

Guía paso a paso: Trabajo colaborativo con GitHub y Git (cmd)

Esta guía explica todo el flujo de trabajo colaborativo con GitHub: desde crear el repositorio del equipo, invitar colaboradores, hasta el ciclo diario de descargar, modificar y subir cambios usando la terminal de Windows (cmd).

Requisitos previos

Antes de empezar, cada integrante del equipo necesita:

1. **Una cuenta de GitHub.** Se crea gratis en <https://github.com/signup>
2. **Git instalado en su computadora.** Se descarga de <https://git-scm.com/downloads>
 - Durante la instalación, dejar todas las opciones por defecto y dar clic en "Siguiente" hasta terminar.
 - Para verificar que quedó instalado, abrir cmd y escribir:

```
git --version
```

Si aparece algo como `git version 2.45.0`, está listo.

3. **Configurar Git con tu identidad** (solo se hace una vez por computadora). En cmd:

```
git config --global user.name "TuUsuarioDeGitHub"  
git config --global user.email "tucorreo@ejemplo.com"
```

Importante: usar el mismo correo con el que te registraste en GitHub. Así los commits quedan asociados a tu cuenta.

PARTE 1: Crear el repositorio del equipo (lo hace UNA persona)

Un solo integrante del equipo crea el repositorio. Los demás serán invitados como colaboradores.

Paso 1. Crear el repositorio en GitHub

1. Iniciar sesión en <https://github.com>
2. En la esquina superior derecha, dar clic en el botón **+** y elegir **Nuevo repositorio** (New repository).
3. Llenar el formulario:
 - **Nombre del repositorio:** por ejemplo Proyecto1_Colaborativo (sin espacios; usar guiones o guiones bajos).
 - **Descripción (opcional):** para qué sirve el proyecto.

- **Visibilidad:** elegir **Público** (cualquiera puede verlo) o **Privado** (solo el equipo).
 - Marcar la casilla **Agregar un archivo README** (Léame). Esto crea el repositorio con un primer archivo y facilita clonarlo después.
4. Dar clic en **Crear repositorio**.

Paso 2. Invitar a los colaboradores

Este paso es **obligatorio**: si no se invita a los compañeros, ellos podrán descargar el proyecto pero **no podrán subir cambios**.

1. Dentro del repositorio, ir a la pestaña **Ajustes** (Settings).
2. En el menú lateral izquierdo, dar clic en **Colaboradores** (Collaborators).
3. GitHub puede pedir la contraseña por seguridad.
4. Dar clic en el botón verde **Agregar personas** (Add people).
5. Escribir el **nombre de usuario de GitHub** del compañero (o su correo) y seleccionarlo.
6. Dar clic en **Agregar al repositorio**.
7. Repetir para cada integrante del equipo.

Cada compañero recibirá una **invitación por correo electrónico** y también una notificación en GitHub.

Paso 3. El colaborador acepta la invitación

Cada compañero invitado debe:

1. Revisar su correo (o las notificaciones de GitHub, el icono de campana).
2. Abrir la invitación y dar clic en **Aceptar invitación** (Accept invitation).

A partir de ese momento ya tiene permisos para subir cambios al repositorio.

PARTE 2: Descargar el proyecto (clonar) — lo hace CADA colaborador

“Clonar” significa descargar una copia completa del repositorio a tu computadora, conectada con GitHub.

Paso 1. Copiar la dirección del repositorio

1. Entrar al repositorio en GitHub.
2. Dar clic en el botón verde **Código** (Code).
3. En la pestaña **HTTPS**, copiar la dirección. Se ve así:
`https://github.com/GioCoto11/Proyecto1_Colaborativo.git`

Paso 2. Clonar desde cmd

1. Abrir cmd (tecla Windows, escribir cmd, Enter).
2. Ir a la carpeta donde querés guardar el proyecto, por ejemplo Documentos:

`cd Documents`

3. Clonar el repositorio (pegar la dirección copiada):

```
git clone https://github.com/GioCoto11/Proyecto1_Colaborativo.git
```

4. Entrar a la carpeta que se acaba de crear:

`cd Proyecto1_Colaborativo`

5. Ver los archivos descargados:

`dir`

¿Dónde quedó la carpeta? Dentro de la carpeta donde estabas parado al hacer el clone. En este ejemplo:

C:\Users\TuUsuario\Documents\Proyecto1_Colaborativo. Para abrirla en el Explorador de Windows desde cmd, escribí: `explorer .`

Este paso solo se hace UNA vez. Después de clonado, el proyecto se mantiene actualizado con otros comandos (ver la Parte 3).

PARTE 3: El ciclo de trabajo diario (bajar → modificar → subir)

Este es el corazón del trabajo colaborativo. Cada vez que vayas a trabajar, seguí este orden **siempre**:

Paso 1. Abrir cmd y entrar a la carpeta del proyecto

`cd Documents\Proyecto1_Colaborativo`

Paso 2. Descargar los cambios más recientes de tus compañeros (PULL)

Siempre antes de empezar a trabajar:

```
git pull
```

Esto trae a tu computadora todo lo que tus compañeros subieron desde la última vez. Si nadie subió nada, dirá `Already up to date` (ya está actualizado).

Regla de oro: hacer `git pull` ANTES de empezar a trabajar y ANTES de subir. Así se evitan la mayoría de los conflictos.

Paso 3. Trabajar en los archivos

Editar los archivos normalmente con tu editor (por ejemplo Visual Studio Code). Si usás VS Code, podés abrir todo el proyecto desde cmd con:

Desarrollo de Páginas Web

Módulo 3: Codificación de Páginas Web

Facilitador: Giovanni Antonio Coto Calderón



code .

Podés crear archivos nuevos, modificar los existentes o borrar los que ya no sirvan.

Paso 4. Revisar qué cambiaste (STATUS)

`git status`

Este comando muestra: - En **rojo**: archivos modificados que todavía no están preparados para subir. - En **verde**: archivos ya preparados (después del `git add`).

Es buena costumbre usarlo antes y después de cada paso para saber en qué estado está todo.

Paso 5. Preparar los cambios (ADD)

`git add .`

El punto (.) significa “todos los archivos modificados”. Si solo querés preparar un archivo específico:

`git add contacto.html`

Paso 6. Guardar los cambios con un mensaje (COMMIT)

`git commit -m "Descripción clara de lo que hiciste"`

Ejemplos de buenos mensajes:

`git commit -m "Agrego formulario a la pagina de contacto"`

`git commit -m "Corrijo enlaces rotos del menu"`

`git commit -m "Estructura inicial del proyecto"`

El commit es como una “fotografía” de tu trabajo en ese momento, con tu nombre y la fecha. El mensaje ayuda a tus compañeros a entender qué hiciste sin tener que revisar el código.

Paso 7. Subir los cambios a GitHub (PUSH)

`git push`

Esto envía tus commits a GitHub para que todo el equipo los vea y pueda descargarlos con `git pull`.

La primera vez que hagás `git push`, Windows abrirá una ventana para iniciar sesión en GitHub con el navegador. Iniciás sesión una vez y queda guardado para siempre.

Resumen del ciclo completo

<code>cd Documents\Proyecto1_Colaborativo</code>	(entrar al proyecto)
<code>git pull</code>	(bajar lo nuevo)
...trabajar en los archivos...	
<code>git status</code>	(revisar qué cambié)
<code>git add .</code>	(preparar los cambios)

```
git commit -m "mensaje descriptivo"    (guardarlos  
con mensaje)                           (bajar por si alguien subió  
git pull                               (subirlos a GitHub)  
mientras trabajaba)
```

PARTE 4: Comandos útiles para revisar el proyecto

Comando	Para qué sirve
git status	Ver qué archivos modificaste y su estado
git log --oneline	Ver el historial de commits en una línea cada uno
git log	Ver el historial completo (autor, fecha, mensaje)
git pull	Descargar los cambios más recientes
git push	Subir tus commits a GitHub
git diff	Ver exactamente qué líneas cambiaste antes del add
cd (solo)	Ver en qué carpeta estás parado
dir	Ver los archivos de la carpeta actual
explorer .	Abrir la carpeta actual en el Explorador de Windows

Para salir de la vista de git log si ocupa toda la pantalla, presionar la tecla **Q**.

PARTE 5: Problemas comunes y cómo resolverlos

“El push fue rechazado” (rejected)

Pasa cuando un compañero subió cambios y vos no los habías descargado. Solución:

```
git pull  
git push
```

Conflicto de fusión (merge conflict)

Ocurre cuando dos personas modificaron **las mismas líneas del mismo archivo**. Al hacer git pull, Git avisa que hay conflicto y marca el archivo así:

```
<<<<<< HEAD  
tu versión de la línea  
=====  
la versión de tu compañero  
>>>>>> abc1234
```

Cómo resolverlo:

1. Abrir el archivo en tu editor.
2. Decidir qué versión dejar (la tuya, la de tu compañero, o una combinación de ambas).
3. Borrar las marcas <<<<<<, ===== y >>>>>>.
4. Guardar el archivo y luego:

```
git add .  
git commit -m "Resuelvo conflicto en contacto.html"  
git push
```

Cómo evitarlos: hacer `git pull` con frecuencia, avisarse entre compañeros qué archivos está tocando cada uno, y de ser posible repartirse archivos distintos.

“No tengo permiso para hacer push” (permission denied / 403)

Significa que no fuiste agregado como colaborador o no aceptaste la invitación. Pedile al dueño del repositorio que revise **Ajustes → Colaboradores** y revisá tu correo para aceptar la invitación.

Hice cambios en un archivo y quiero descartarlos (volver a como estaba)

```
git checkout -- nombre_del_archivo.html
```

Cuidado: esto borra tus cambios no guardados en ese archivo y no se puede deshacer.

PARTE 6: Trabajar con ramas (branches) — una rama por colaborador

Las ramas permiten que cada colaborador trabaje en su propia “copia paralela” del proyecto sin afectar la rama principal (main o master) hasta que su trabajo esté listo y revisado. Es la forma más ordenada y profesional de colaborar.

Conceptos básicos de ramas

Comando	Para qué sirve
<code>git branch</code>	Ver las ramas existentes; la actual aparece con un asterisco (*)
<code>git checkout -b nombre-rama</code>	Crear una rama nueva y cambiarte a ella
<code>git checkout nombre-rama</code>	Cambiarte a una rama que ya existe
<code>git push -u origin nombre-rama</code>	Subir la rama a GitHub por primera vez
<code>git branch -d nombre-rama</code>	Borrar una rama local que ya se fusionó

Nota: la rama principal puede llamarse main o master según cómo se creó el repositorio. Verificalo con `git branch`. En los ejemplos usaremos main; si tu repositorio usa master, solo cambiá el nombre en los comandos.

EJEMPLO PRÁCTICO: 3 colaboradores, 3 páginas, 3 ramas

El escenario: el equipo tiene tres integrantes (Ana, Beto y Carla) y el sitio web tiene tres páginas. Se reparten así:

Colaborador	Página a cargo	Nombre de su rama
Ana	index.html	pagina-index
Beto	nosotros.html	pagina-nosotros
Carla	contacto.html	pagina-contacto

Todos parten del mismo repositorio ya clonado (Parte 2). Los pasos son idénticos para los tres, solo cambia el nombre de la rama y el archivo. Los mostramos con Ana.

Paso 1. Ana se asegura de partir de lo más reciente

```
cd Documents\Proyecto1_Colaborativo
git checkout main
git pull
```

Esto es clave: la rama nueva debe nacer de la versión más actualizada de main.

Paso 2. Ana crea su rama y se cambia a ella

```
git checkout -b pagina-index
```

Confirmá que estás en la rama correcta:

```
git branch
```

Debería mostrar:

```
main
* pagina-index
```

Paso 3. Ana trabaja en su archivo

Edita index.html con su editor, agrega el contenido, guarda. **Todo lo que haga aquí solo existe en su rama;** la rama main y los archivos de sus compañeros no se tocan.

Paso 4. Ana guarda y sube su rama

```
git add index.html
git commit -m "Agrego contenido de la pagina principal"
git push -u origin pagina-index
```

El -u origin pagina-index solo es necesario la **primera vez** que sube la rama; después basta con git push.

Mientras tanto, Beto y Carla hacen exactamente lo mismo con sus ramas:

Desarrollo de Páginas Web

Módulo 3: Codificación de Páginas Web

Facilitador: Giovanni Antonio Coto Calderón



:: Beto

```
git checkout main
git pull
git checkout -b pagina-nosotros
...edita nosotros.html...
git add nosotros.html
git commit -m "Agrego contenido de la pagina nosotros"
git push -u origin pagina-nosotros
```

:: Carla

```
git checkout main
git pull
git checkout -b pagina-contacto
...edita contacto.html...
git add contacto.html
git commit -m "Agrego formulario de contacto"
git push -u origin pagina-contacto
```

Paso 5. Cada uno crea su Pull Request en GitHub

Cuando Ana termina y hace push, al entrar al repositorio en GitHub aparece un aviso amarillo con el botón **Comparar y solicitar extracción** (Compare & pull request) — el mismo que se ve cuando hay “empujes recientes”. Ana da clic y:

1. Verifica que la fusión sea de pagina-index **hacia** main (base: main ← compare: pagina-index).
2. Escribe un título y una breve descripción de lo que hizo.
3. Da clic en **Crear solicitud de extracción** (Create pull request).

Si el botón no aparece, se puede hacer manual: pestaña **Solicitudes de extracción** → **Nueva solicitud de extracción** → elegir las ramas.

Paso 6. Un compañero revisa y fusiona

Otro integrante (por ejemplo Beto) abre la solicitud de Ana, revisa los cambios en la pestaña **Archivos cambiados**, y si todo está bien da clic en **Fusionar solicitud de extracción** (Merge pull request) → **Confirmar fusión**. Los cambios de Ana ya forman parte de main.

Se repite lo mismo con las solicitudes de Beto y Carla. Como cada uno trabajó en un **archivo distinto**, las tres fusiones entran sin ningún conflicto.

Paso 7. Todos actualizan su copia local con el proyecto unido

Después de fusionar las tres ramas, cada colaborador ejecuta:

```
git checkout main
git pull
```

Desarrollo de Páginas Web

Módulo 3: Codificación de Páginas Web

Facilitador: Giovanni Antonio Coto Calderón

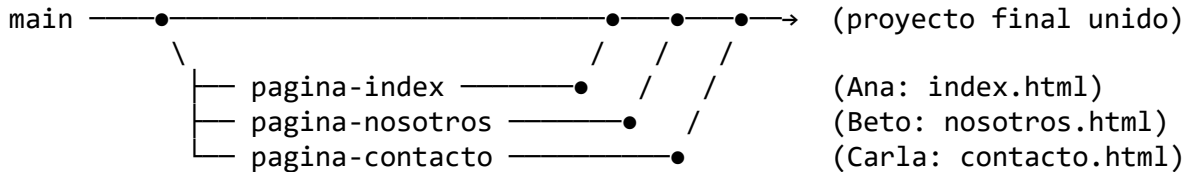


Ahora los tres tienen en su computadora el proyecto final completo: `index.html` de Ana, `nosotros.html` de Beto y `contacto.html` de Carla, todo unido en `main`.

Opcionalmente, cada quien puede borrar su rama local que ya cumplió su propósito:

```
git branch -d pagina-index
```

Resumen visual del flujo



PARTE 7: Dos colaboradores en el MISMO archivo (`index.html`)

Este es el caso más delicado del trabajo colaborativo: dos personas editando el mismo archivo. Veamos cómo hacerlo bien.

El escenario: Ana y Beto van a trabajar los dos sobre `index.html`, pero en secciones diferentes:

Colaborador	Sección de <code>index.html</code>	Nombre de su rama
Ana	El header (encabezado y menú)	<code>index-header</code>
Beto	El footer (pie de página)	<code>index-footer</code>

Punto de partida: para que esto funcione bien, `index.html` ya debe existir en `main` con una estructura base donde cada sección tenga su lugar. Por ejemplo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Proyecto Colaborativo</title>
</head>
<body>

  <header>
    <!-- Aquí trabaja Ana -->
  </header>

  <main>
    <p>Contenido principal del sitio.</p>
  </main>
```

```
<footer>
  <!-- Aquí trabaja Beto -->
</footer>

</body>
</html>
```

Esa estructura base la sube una persona a main primero (con el flujo normal: add, commit, push). Tener “espacios reservados” bien separados reduce muchísimo los conflictos.

Paso 1. Cada uno crea su rama desde main actualizado

```
:: Ana
git checkout main
git pull
git checkout -b index-header
```

```
:: Beto (en su computadora)
git checkout main
git pull
git checkout -b index-footer
```

Paso 2. Cada uno edita SOLO su sección

Ana modifica únicamente lo que está dentro de <header>...</header>:

```
<header>
  <h1>Proyecto Colaborativo</h1>
  <nav>
    <a href="index.html">Inicio</a>
    <a href="nosotros.html">Nosotros</a>
    <a href="contacto.html">Contacto</a>
  </nav>
</header>
```

Beto modifica únicamente lo que está dentro de <footer>...</footer>:

```
<footer>
  <p>&copy; 2026 Equipo Proyecto1. Todos los derechos reservados.</p>
  <p>Contacto: equipo@proyecto1.com</p>
</footer>
```

La disciplina es la clave: cada quien toca solo su sección. Si Ana “de paso” corrige algo en el footer, ahí es donde nacen los conflictos.

Paso 3. Cada uno sube su rama

```
:: Ana
git add index.html
git commit -m "Agrego header con menu de navegacion"
```

Desarrollo de Páginas Web

Módulo 3: Codificación de Páginas Web

Facilitador: Giovanni Antonio Coto Calderón



```
git push -u origin index-header
```

```
:: Beto
git add index.html
git commit -m "Agrego footer con derechos y contacto"
git push -u origin index-footer
```

Paso 4. Fusionar la primera rama (sin problema)

Ana crea su Pull Request (index-header → main) y el equipo la fusiona. Hasta aquí todo normal: main ya tiene el header de Ana.

Paso 5. Fusionar la segunda rama — el momento clave

Cuando Beto crea su Pull Request (index-footer → main), pueden pasar dos cosas:

Caso A — Se fusiona automáticamente (lo más común si trabajaron ordenado).

Como Ana tocó las líneas del <header> y Beto las del <footer>, Git es capaz de combinar ambos cambios solo. GitHub muestra “Able to merge” (se puede fusionar) en verde, se fusiona normal, y **el index.html final queda con el header de Ana Y el footer de Beto, unidos en un único archivo**. Nadie tuvo que copiar y pegar nada.

Caso B — Conflicto (si ambos tocaron líneas iguales o muy cercanas).

GitHub muestra “This branch has conflicts that must be resolved” (esta rama tiene conflictos que deben resolverse). Beto lo resuelve desde cmd así:

1. En su rama, traer lo más reciente de main (que ya incluye el header de Ana):

```
git checkout index-footer
git pull origin main
```

2. Git avisa del conflicto y marca index.html con las señales de conflicto:

```
<<<<<<< HEAD
      (la versión de la rama de Beto)
=====
      (la versión que ya está en main, con lo de Ana)
>>>>>>> main
```

3. Beto abre index.html en su editor y **arma la versión final combinando ambas partes**: deja el header de Ana Y su footer, y borra las tres líneas de marcas (<<<<<<<, =====, >>>>>>>).
4. Guarda y sube la resolución:

```
git add index.html
git commit -m "Resuelvo conflicto: uno header y footer en index.html"
git push
```

5. El Pull Request en GitHub se actualiza solo, ahora sí muestra que se puede fusionar, y el equipo lo fusiona.

Paso 6. Todos descargan el archivo final unido

```
git checkout main  
git pull
```

Al abrir index.html, cada integrante verá **un único archivo** con el header de Ana, el contenido principal y el footer de Beto: el trabajo de ambos, unido.

Reglas de oro para compartir un mismo archivo

1. **Partir de una estructura base común** ya subida a main, con las secciones bien separadas.
2. **Cada quien edita solo su sección** — no “corregir de paso” lo del compañero.
3. **Ramas cortas:** terminar, subir y fusionar rápido. Cuanto más tiempo vive una rama, más se aleja de main y más probables son los conflictos.
4. **Fusionar de uno en uno:** primero un Pull Request, luego el otro. No fusionar dos a la vez.
5. **Avisarse:** un mensaje de “voy a tocar el header de index” evita sorpresas.

Glosario rápido

Término	Significado
Repositorio (repo)	La carpeta del proyecto con todo su historial de cambios
Clonar (clone)	Descargar una copia completa del repositorio por primera vez
Commit	Una “fotografía” guardada de tus cambios, con mensaje, autor y fecha
Push (empujar)	Subir tus commits a GitHub
Pull (jalar)	Descargar los commits nuevos de tus compañeros
Rama (branch/sucursal)	Línea de trabajo paralela dentro del proyecto
Merge (fusionar)	Unir los cambios de una rama con otra
Conflicto	Cuando dos personas cambiaron las mismas líneas y Git necesita que decidas cuál versión queda
Pull Request (solicitud de extracción)	Propuesta para incorporar los cambios de una rama a la principal, con revisión del equipo